
Laboratorio integrador

Ejercicio 1: Construcción de un Servidor de Consulta (GET)

En este ejercicio vamos a crear un "Directorio de Contactos" muy simple para entender cómo el servidor entrega información al cliente.

Paso 1.1: El Cascarón

Abran su editor (VS Code) y creen un archivo llamado `server.js`. Escriban lo siguiente para levantar los cimientos:

```
const express = require('express');
const app = express();

// Definimos el puerto
const PORT = 3000;

app.listen(PORT, () => {
  console.log("Servidor activo en el puerto " + PORT);
});
```

Práctica: Corran en su terminal `node server.js`. Si ven el mensaje de "Servidor activo", ¡vamos por buen camino!

Paso 1.2: El Endpoint de saludo

Ahora, vamos a crear la ruta. Justo antes del `app.listen`, agreguen estas líneas. **Escríbanlo manualmente**, no lo copien:

```
app.get('/hola', (req, res) => {
  res.send("<h1>¡Hola Estudiantes!</h1><p>El servidor respond");
});
```

Práctica: Guarden, reinicien el servidor (Ctrl+C y vuelvan a ejecutar `node server.js`) y abran su navegador en `http://localhost:3000/hola`. ¿Ven el texto en grande?

Paso 1.3: Devolviendo Datos Reales (JSON)

En el mundo real, los servidores no suelen enviar HTML, sino **JSON**. Modifiquen su ruta para que se vea así:

```
app.get('/contacto', (req, res) => {  
  res.json({  
    nombre: "Soporte Técnico",  
    email: "ayuda@irsi.com",  
    extension: 2205  
  });  
});
```

Reto en Postman: Abran Postman, hagan un **GET** a `http://localhost:3000/contacto`. Verifiquen que abajo diga **Status: 200 OK**.

Ejercicio 2: El Receptor de Datos (POST)

Ahora vamos a simular que un estudiante se registra en una lista. Aquí es donde usamos el "Cuerpo" (Body) de la petición.

Paso 2.1: El Traductor (Middleware)

Si enviamos datos y no le decimos a Express cómo leerlos, llegarán como `undefined`. Agreguen esta línea **arriba de todas sus rutas**:

```
app.use(express.json());
```

Paso 2.2: Creando la ruta de Registro

Vamos a recibir un nombre y un carné. Escriban esto debajo de sus rutas GET:

```
app.post('/registrar', (req, res) => {  
  // Extraemos la info del body  
  const infoEstudiante = req.body;  
  
  console.log("Datos recibidos:", infoEstudiante);  
  
  res.status(201).json({  
    mensaje: "Estudiante registrado con éxito",  
    datos_recibidos: infoEstudiante  
  });  
});
```

Paso 2.3: La prueba de fuego en Postman (Siguan estos pasos exactos)

1. Cambien el método a **POST** en Postman.
2. Pongan la URL: `http://localhost:3000/registrar`.
3. Vayan a la pestaña **Body**.
4. Seleccionen el botoncito **raw**.
5. Al final de esa fila, cambien "Text" por **JSON**.
6. Escriban este objeto:

```
{  
  "nombre": "Su Nombre",  
  "carne": "C12345"  
}
```

7. Denle a **Send**. Si el servidor les responde con los mismos datos y un código **201**, ¡lo lograron!

Ejercicio 3: El Motor de Calificaciones (Middleware y Callbacks)

Debes crear un servidor Express que reciba una lista de estudiantes con sus notas, calcule quiénes aprobaron y quiénes no, y devuelva un reporte estructurado.

Paso a paso para los estudiantes:

1. **Configuración:** Inicia un proyecto con `npm init -y` e instala express.
2. **La Lógica del Motor:** Crea una función llamada `procesarEstudiantes(lista, callback)`.
 - Esta función debe usar el patrón **Error-First**.

- Si la lista está vacía o no es un arreglo, debe retornar el error al callback.
- Si es válida, debe usar `forEach` para clasificar a los estudiantes en dos grupos: aprobados (nota ≥ 70) y reprobados (nota < 70).

3. **La Ruta de Express:** Crea una ruta `POST /analizar-clase`.

- Debe recibir un JSON con la propiedad `estudiantes` (un arreglo de objetos).
- Debe llamar a la función `procesarEstudiantes`.
- Debe responder con el reporte generado.

Ejemplo de Interacción (Input JSON):

```
{
  "estudiantes": [
    { "nombre": "Diego", "nota": 85 },
    { "nombre": "Ana", "nota": 60 },
    { "nombre": "Carlos", "nota": 92 }
  ]
}
```

Salida Esperada (Output JSON):

```
{
  "status": "success",
  "reporte": {
    "aprobados": ["Diego", "Carlos"],
    "reprobados": ["Ana"],
    "total": 3
  }
}
```

Ejercicio 4: Enunciado de Análisis - Sistema de Logística "Aethelgard Express"

La estación espacial necesita un sistema que reciba pedidos de suministros desde la terminal de comandos (Node.js) y los registre en un servidor central (Express).

Especificaciones del Sistema:

1. El Servidor (Express - `server.js`):

- Cree un servidor que escuche en el puerto 4000.
- Implemente una ruta `POST /despacho`.

- **Lógica Asíncrona:** El servidor debe simular que verifica el espacio en el hangar con un `setTimeout` de 1.5 segundos.
- **Regla de Negocio:** Si el peso del envío (peso) es mayor a 500kg, debe responder con un error 400 y el mensaje "Capacidad de hangar excedida".
- Si es menor, debe devolver un JSON confirmando el registro del paquete.

2. El Cliente de Terminal (Node.js - cliente.js):

- El script debe capturar el **nombre del recurso** y el **peso** desde `process.argv`.
- **Lógica de Callback:** Escriba una función `prepararPaquete(nombre, peso, callback)` que use una función de flecha para imprimir: "Empacando [recurso]...".
- Luego, debe mostrar en consola: "Enviando datos al servidor central..."

3. Análisis de Datos:

- Usted debe decidir cómo estructurar el objeto JSON que se envía.
- Debe manejar el caso en que el usuario no pase los argumentos en la terminal.

Ejemplo de Interacción (Terminal):

```
node cliente.js "Oxígeno" 350
```

Flujo de Salida esperado:

1. Terminal: 1. Empacando Oxígeno...
2. Terminal: 2. Enviando datos al servidor central...
3. Servidor (tras 1.5s): [LOG] Recibido pedido de Oxígeno (350kg). Estado: ACEPTADO.
4. Terminal: Respuesta del Servidor: { "mensaje": "Despacho programado", "id": "A-102" }

Ejercicio 5: Sistema de Filtros "IRSI Talent" (GET con Query Params)

Contexto: La empresa IRSI necesita un buscador rápido para su departamento de Recursos Humanos. En lugar de recibir datos en el cuerpo (body), los recibiremos directamente desde la URL.

Especificaciones del Reto:

1. **La Ruta:** Crea una ruta GET `/buscar`.

2. **Captura de Parámetros:** El servidor debe ser capaz de recibir dos parámetros por la URL: nombre y rol.

3. **Lógica de Respuesta:**

- Si el usuario envía ambos datos (ejemplo: ?nombre=Diego&rol=Analista), el servidor debe responder con un status **200** y un mensaje que combine ambos: "Resultado: Se ha encontrado al Analista Diego en la base de datos."
- Si el usuario no envía el nombre, el servidor debe responder con un status **400** y el mensaje: "Error: El parámetro 'nombre' es obligatorio para la búsqueda."

Cómo deben probarlo los estudiantes:

En Postman, no usarán la pestaña "Body". Deben escribir la ruta directamente en la barra de direcciones o usar la pestaña **Params**:

Ejemplo de URL para probar: `http://localhost:3000/buscar?nombre=Ana&rol=Programadora`

¿Qué deben aplicar?

- Uso de req.query para extraer los datos.
- Validación simple con if para verificar si el nombre existe.
- Manejo de códigos de estado HTTP correctos.

Rúbrica de Evaluación

Criterio	Nivel 4: Excelente (100%)	Nivel 3: Bueno (80%)	Nivel 2: Necesita Mejora (60%)	Nivel 1: Insuficiente (0-40%)
Configuración y GET (E1)	El servidor levanta sin errores, usa el puerto correcto y los endpoints /hola y /contacto devuelven HTML y JSON exactos.	El servidor funciona pero tiene errores menores en la estructura del JSON o formatos de texto.	El servidor levanta, pero solo uno de los endpoints funciona correctamente.	El servidor no levanta o no implementa las rutas solicitadas.
Manejo de POST y Body (E2)	Implementa correctamente el middleware express.json()	Usa el método POST y recibe datos, pero olvida el	No usa el middleware de JSON, por lo que recibe	No implementa la ruta POST o no entiende

	y captura el req.body. El endpoint /registrar responde con status 201 .	status 201 o no devuelve el JSON de confirmación.	datos undefined o la ruta tiene errores de sintaxis.	cómo acceder al cuerpo de la petición.
Lógica de Callbacks (E3)	Implementa la función con el patrón Error-First . La lógica de forEach separa correctamente aprobados de reprobados.	Implementa la lógica de filtrado pero no utiliza el patrón Error-First correctamente en el callback.	El filtrado de notas tiene errores lógicos (confunde ≥ 70 con > 70) o no usa el callback solicitado.	No logra separar a los estudiantes o no utiliza una función de callback para procesar los datos.
Asincronía y Cliente (E4)	Logra la comunicación entre cliente.js y server.js. Usa setTimeout de forma lógica y valida el peso ($> 500\text{kg}$) con status 400 .	Existe comunicación pero no maneja el retardo asíncrono o la validación del peso no envía el código de error correcto.	El cliente envía los datos pero el servidor no procesa la lógica de negocio o falta el uso de process.argv.	No existe comunicación entre archivos o no se implementó la lógica asíncrona solicitada.
Uso de Query Params (E5)	Captura nombre y rol vía req.query. Valida la existencia del nombre y maneja status 200 y 400 según corresponda.	Captura los parámetros pero no realiza la validación de obligatoriedad del nombre o no usa los status codes.	Confunde req.query con req.body o la respuesta no es dinámica según los parámetros.	No implementa la ruta de búsqueda o no logra extraer datos de la URL.
Buenas Prácticas	El código está indentado, usa nombres de variables claros (camelCase), funciones flecha y no tiene errores en consola.	El código funciona pero la indentación es pobre o los nombres de las variables son genéricos (x, y, data1).	Mezcla estilos de programación (var y const) o deja mucho código comentado innecesario.	El código es desordenado, difícil de leer o presenta errores constantes por mala sintaxis.